

JOB SHEET 12

Praktikum

Dosen Pengampu:
Mungki Astiningrum, ST, M.Kom



Disusun oleh:
Mukhammad Raffi Zabra

T1 – 1G
254107020059

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

65141

TAHUN 2025-2026

12.2 Percobaan 1: Operasi Penambahan pada Double Linked List

12.2.1 Langkah-Langkah Percobaan

- **Mahasiswa24.java**

Code :

```
package jobsheet12;

public class mahasiswa24 {
    String nim, nama, kelas;
    double ipk;

    public mahasiswa24() {
    }

    public mahasiswa24(String nim, String nama, String kelas, double ipk){
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    public void tampil(){
        System.out.println("nim : " + nim + "\nnama : " + nama+ "\nkelas : " + kelas + "\nipk : " + ipk);
    }
}
```

- **Node24.java**

Code :

```
package jobsheet12;

public class node24 {
    mahasiswa24 data;
    node24 prev;
    node24 next;

    public node24(mahasiswa24 data){
        this.data = data;
        this.prev = null;
        this.next = null;
    }
}
```

- **Doublelinkedlist24.java**

Code :

```
package jobsheet12;

import java.util.Scanner;

public class doublelinkedlist24 {
    node24 head;
    node24 tail;

    public doublelinkedlist24() {
        this.head = null;
        this.tail = null;
    }
}
```

```

}

public boolean isEmpty() {
    return head == null;
}

public void addfirst(mahasiswa24 data) {
    node24 newnode = new node24(data);
    if (isEmpty()) {
        head = tail = newnode;
    } else {
        newnode.next = head;
        head.prev = newnode;
        head = newnode;
    }
}

public void addLast(mahasiswa24 data) {
    node24 newnode = new node24(data);
    if (isEmpty()) {
        head = tail = newnode;
    } else {
        tail.next = newnode;
        newnode.prev = tail;
        tail = newnode;
    }
}

public void insertAfter(String keyNim, mahasiswa24 data) {
    node24 current = head;
    while (current != null && !current.data.nim.equals(keyNim)) {
        current = current.next;
    }
    if (current == null) {
        System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
        return;
    }

    node24 newnode = new node24(data);

    if (current == tail) {
        newnode.prev = current;
        current.next = newnode;
        tail = newnode;
    } else {
        newnode.prev = current;
        newnode.next = current.next;
        current.next.prev = newnode;
        current.next = newnode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}

public void print(){
    if (isEmpty()){

```

```

        System.out.println("linked list masih kosong");
        return;
    }

    node24 current = head;
    while(current != null){
        current.data.tampil();
        current = current.next;
    }
}

// method ini saya buat sendiri karna di langkah" belumn ada
public static mahasiswa24 inputMahasiswa(Scanner scan) {
    System.out.print("Masukkan NIM: ");
    String nim = scan.nextLine();
    System.out.print("Masukkan Nama: ");
    String nama = scan.nextLine();
    System.out.print("Masukkan Kelas: ");
    String kelas = scan.nextLine();
    System.out.print("Masukkan IPK: ");
    double ipk = scan.nextDouble();
    scan.nextLine();

    return new mahasiswa24(nim, nama, kelas, ipk);
}

public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);
    doublelinkedlist24 list = new doublelinkedlist24();
    int pilihan;

    do {
        System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
        System.out.println("1. Tambah data di awal");
        System.out.println("2. Tambah data di akhir");
        System.out.println("3. Sisipkan data di tengah (setelah NIM)");
        System.out.println("4. Hapus data di awal");
        System.out.println("5. Hapus data di akhir");
        System.out.println("6. Tampilkan data");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu : ");
        pilihan = scan.nextInt();
        scan.nextLine();

        switch (pilihan) {
            case 1:
                mahasiswa24 mhsAwal = inputMahasiswa(scan);
                list.addfirst(mhsAwal);
                break;
            case 2:
                mahasiswa24 mhsAkhir = inputMahasiswa(scan);
                list.addLast(mhsAkhir);
                break;
            case 3:
                System.out.print("Masukkan NIM yang dicari : ");

```

```

        String keyNim = scan.nextLine();
        System.out.println("Masukkan data baru: ");
        mahasiswa24 dataBaru = inputMahasiswa(scan);
        list.insertAfter(keyNim, dataBaru);
        break;
    case 4:
        System.out.println("hapus data di percobaan 2");
        break;
    case 5:
        System.out.println("hapus data di percobaan 2");
        break;
    case 6:
        list.print();
        break;
    case 0:
        System.out.println("Program selesai.");
        break;
    default:
        System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}
}

```

Hasil :

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 2
Masukkan NIM: 123005
Masukkan Nama: harry
Masukkan Kelas: 1A
Masukkan IPK: 3,76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 3
Masukkan NIM yang dicari : 123005
Masukkan data baru:
Masukkan NIM: 123010
Masukkan Nama: potter
Masukkan Kelas: 1B
Masukkan IPK: 3,55
Data berhasil disisipkan setelah NIM 123005

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 6
nim : 123005
nama : harry
kelas : 1A
ipk : 3.76
nim : 123010
nama : potter
kelas : 1B
ipk : 3.55

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
0. Keluar
Pilih menu : 0
Program selesai.

```

12.2.3 Pertanyaan

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

Jawaban :

Struktur:

- Single Linked List: Setiap *node* hanya memiliki satu *pointer* (penunjuk), yaitu *next* yang menunjuk ke *node* berikutnya.
- Double Linked List : Setiap *node* memiliki dua *pointer*, yaitu *next* menunjuk ke *node* berikutnya dan *prev* menunjuk ke *node* sebelumnya.

Mekanisme Traversal:

- Single Linked List: Traversal (penelusuran) hanya bisa dilakukan satu arah, yaitu maju dari head (awal) menuju tail (akhir).
- Double Linked List: Traversal bisa dilakukan dua arah (*bidirectional*). Kamu bisa menelusuri maju dari head ke tail (menggunakan *next*), dan juga mundur dari tail ke head (menggunakan *prev*).

2. Perhatikan class Node, di dalamnya terdapat atribut *next* dan *prev*. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!

Jawaban :

- Atribut *next*: Berfungsi untuk menyimpan referensi (alamat memori) dari *node* setelahnya. Saat traversal, *next* digunakan untuk maju ke data berikutnya. Saat manipulasi seperti *insert*, *next* digunakan untuk menyambungkan *node* saat ini dengan *node* di depannya.
- Atribut *prev*: Berfungsi untuk menyimpan referensi dari *node* sebelumnya. Saat traversal, *prev* digunakan untuk mundur ke data sebelumnya. Saat manipulasi, *prev* memastikan rantai *node* tidak terputus dari arah belakang.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

Jawaban :

Fungsinya adalah untuk menciptakan kondisi awal di mana linked list benar-benar kosong. Saat objek *double linked list* pertama kali dibuat, belum ada *node* (data) apa pun di dalamnya, sehingga penunjuk ke awal list (head) dan penunjuk ke akhir list (tail) tidak menunjuk ke alamat memori manapun (*null*).

4. Perhatikan potongan kode berikut:

```
if (isEmpty()) {  
    head = tail = newNode;  
}
```

Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

Jawaban :

Ketika *linked list* masih kosong dan ada satu node baru yang dimasukkan, maka node tersebut otomatis menjadi satu-satunya elemen di dalam *list*.

Karena dia adalah satu-satunya data, maka node tersebut berstatus sebagai data yang paling depan (head) sekaligus data yang paling belakang (tail). Oleh karena itu, kedua *pointer* tersebut harus menunjuk ke *node* yang sama.

5. Modifikasi method print() agar menampilkan pesan “Linked List masih kosong” ketika tidak terdapat data pada linked list!

Jawaban :

```
public void print(){  
    if (isEmpty()){  
        System.out.println(“linked list masih kosong”);  
        return;  
    }  
}
```

```

    }

    node24 current = head;
    while(current != null){
        current.data.tampil();
        current = current.next;
    }
}

```

6. Modifikasi kode program dengan menambahkan method printReverse() untuk menampilkan seluruh data pada Double Linked List secara terbalik, dimulai dari node tail menuju head!

Jawaban :

```
package jobsheet12;
```

```
import java.util.Scanner;
```

```
public class doublelinkedlist24 {
    node24 head;
    node24 tail;
```

```

    public doublelinkedlist24() {
        head = null;
        tail = null;
    }

```

```

    public boolean isEmpty() {
        return head == null;
    }

```

```

    public void addfirst(mahasiswa24 data) {
        node24 newnode = new node24(data);
        if (isEmpty()) {
            head = tail = newnode;
        } else {
            newnode.next = head;
            head.prev = newnode;
            head = newnode;
        }
    }

```

```

    public void addLast(mahasiswa24 data) {
        node24 newnode = new node24(data);
        if (isEmpty()) {
            head = tail = newnode;
        } else {
            tail.next = newnode;
            newnode.prev = tail;
            tail = newnode;
        }
    }

```

```

    public void insertAfter(String keyNim, mahasiswa24 data) {
        node24 current = head;
        while (current != null && !current.data.nim.equals(keyNim)) {

```

```

        current = current.next;
    }
    if (current == null) {
        System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
        return;
    }

    node24 newnode = new node24(data);

    if (current == tail) {
        newnode.prev = current;
        current.next = newnode;
        tail = newnode;
    } else {
        newnode.prev = current;
        newnode.next = current.next;
        current.next.prev = newnode;
        current.next = newnode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}

public void print(){
    if (isEmpty()){
        System.out.println("linked list masih kosong");
        return;
    }

    node24 current = head;
    while(current != null){
        current.data.tampil();
        current = current.next;
    }
}

public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong");
        return;
    }

    System.out.println("Menampilkan data dari belakang (Tail ke Head):");
    node24 current = tail;

    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

// method ini saya buat sendirti karna di langkah" belumn ada
public static mahasiswa24 inputMahasiswa(Scanner scan) {
    System.out.print("Masukkan NIM: ");
    String nim = scan.nextLine();
    System.out.print("Masukkan Nama: ");

```

```

String nama = scan.nextLine();
System.out.print("Masukkan Kelas: ");
String kelas = scan.nextLine();
System.out.print("Masukkan IPK: ");
double ipk = scan.nextDouble();
scan.nextLine();

return new mahasiswa24(nim, nama, kelas, ipk);
}

public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);
    doublelinkedlist24 list = new doublelinkedlist24();
    int pilihan;

    do {
        System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
        System.out.println("1. Tambah data di awal");
        System.out.println("2. Tambah data di akhir");
        System.out.println("3. Sisipkan data di tengah (setelah NIM)");
        System.out.println("4. Hapus data di awal");
        System.out.println("5. Hapus data di akhir");
        System.out.println("6. Tampilkan data");
        System.out.println("7. Tampilkan data dari belakang");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu : ");
        pilihan = scan.nextInt();
        scan.nextLine();

        switch (pilihan) {
            case 1:
                mahasiswa24 mhsAwal = inputMahasiswa(scan);
                list.addfirst(mhsAwal);
                break;
            case 2:
                mahasiswa24 mhsAkhir = inputMahasiswa(scan);
                list.addLast(mhsAkhir);
                break;
            case 3:
                System.out.print("Masukkan NIM yang dicari : ");
                String keyNim = scan.nextLine();
                System.out.println("Masukkan data baru: ");
                mahasiswa24 dataBaru = inputMahasiswa(scan);
                list.insertAfter(keyNim, dataBaru);
                break;
            case 4:
                System.out.println("hapus data di percobaan 2");
                break;
            case 5:
                System.out.println("hapus data di percobaan 2");
                break;
            case 6:
                list.print();
                break;
            case 7:

```

```

        list.printReverse();
        break;
    case 0:
        System.out.println("Program selesai.");
        break;
    default:
        System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}
}

```

Hasil :

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data dari belakang
0. Keluar
Pilih menu : 7
Menampilkan data dari belakang (Tail ke Head):
nim : 123010
nama : potter
kelas : 1B
ipk : 3.55
nim : 123005
nama : harry
kelas : 1A
ipk : 3.76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data dari belakang
0. Keluar
Pilih menu : 6
nim : 123005
nama : harry
kelas : 1A
ipk : 3.76
nim : 123010
nama : potter
kelas : 1B
ipk : 3.55

```

12.3 Percobaan 2: Operasi Penghapusan pada Double Linked List

12.3.1 Langkah-Langkah Percobaan

- Doublelinkedlist24.java

Code :

```
package jobsheet12;
```

```
import java.util.Scanner;
```

```

public class doublelinkedlist24 {
    node24 head;
    node24 tail;

    public doublelinkedlist24() {
        head = null;
        tail = null;
    }
}

```

```

public boolean isEmpty() {
    return head == null;
}

public void addfirst(mahasiswa24 data) {
    node24 newNode = new node24(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}

public void addLast(mahasiswa24 data) {
    node24 newNode = new node24(data);
    if (isEmpty()) {
        head = tail = newNode;
    } else {
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}

public void insertAfter(String keyNim, mahasiswa24 data) {
    node24 current = head;
    while (current != null && !current.data.nim.equals(keyNim)) {
        current = current.next;
    }
    if (current == null) {
        System.out.println("Data dengan NIM " + keyNim + " tidak ditemukan.");
        return;
    }

    node24 newNode = new node24(data);

    if (current == tail) {
        newNode.prev = current;
        current.next = newNode;
        tail = newNode;
    } else {
        newNode.prev = current;
        newNode.next = current.next;
        current.next.prev = newNode;
        current.next = newNode;
    }
    System.out.println("Data berhasil disisipkan setelah NIM " + keyNim);
}

public void print(){
    if (isEmpty()){
        System.out.println("linked list masih kosong");
    }
}

```

```

        return;
    }

    node24 current = head;
    while(current != null){
        current.data.tampil();
        current = current.next;
    }
}

public void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked list masih kosong");
        return;
    }

    System.out.println("Menampilkan data dari belakang (Tail ke Head):");
    node24 current = tail;

    while (current != null) {
        current.data.tampil();
        current = current.prev;
    }
}

// method ini saya buat sendiri karna di langkah" belumn ada
public static mahasiswa24 inputMahasiswa(Scanner scan) {
    System.out.print("Masukkan NIM: ");
    String nim = scan.nextLine();
    System.out.print("Masukkan Nama: ");
    String nama = scan.nextLine();
    System.out.print("Masukkan Kelas: ");
    String kelas = scan.nextLine();
    System.out.print("Masukkan IPK: ");
    double ipk = scan.nextDouble();
    scan.nextLine();

    return new mahasiswa24(nim, nama, kelas, ipk);
}

public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
    System.out.println("Data di awal berhasil dihapus.");
}

```

```

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
    System.out.println("Data di akhir berhasil dihapus.");
}

public static void main(String[] args) {
    Scanner scan = new Scanner(System.in);
    doublelinkedlist24 list = new doublelinkedlist24();
    int pilihan;

    do {
        System.out.println("\n===== MENU DOUBLE LINKED LIST =====");
        System.out.println("1. Tambah data di awal");
        System.out.println("2. Tambah data di akhir");
        System.out.println("3. Sisipkan data di tengah (setelah NIM)");
        System.out.println("4. Hapus data di awal");
        System.out.println("5. Hapus data di akhir");
        System.out.println("6. Tampilkan data");
        System.out.println("7. Tampilkan data dari belakang");
        System.out.println("0. Keluar");
        System.out.print("Pilih menu : ");
        pilihan = scan.nextInt();
        scan.nextLine();

        switch (pilihan) {
            case 1:
                mahasiswa24 mhsAwal = inputMahasiswa(scan);
                list.addfirst(mhsAwal);
                break;
            case 2:
                mahasiswa24 mhsAkhir = inputMahasiswa(scan);
                list.addLast(mhsAkhir);
                break;
            case 3:
                System.out.print("Masukkan NIM yang dicari : ");
                String keyNim = scan.nextLine();
                System.out.println("Masukkan data baru: ");
                mahasiswa24 dataBaru = inputMahasiswa(scan);
                list.insertAfter(keyNim, dataBaru);
                break;
            case 4:
                list.removeFirst();
                break;
            case 5:
                list.removeLast();

```

```

        break;
    case 6:
        list.print();
        break;
    case 7:
        list.printReverse();
        break;
    case 0:
        System.out.println("Program selesai.");
        break;
    default:
        System.out.println("Menu tidak valid.");
    }
} while (pilihan != 0);
scan.close();
}
}

```

Hasil :

```

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data dari belakang
0. Keluar
Pilih menu : 5
Data di akhir berhasil dihapus.

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data dari belakang
0. Keluar
Pilih menu : 6
nim : 123005
nama : harry
kelas : 1A
ipk : 3.76

===== MENU DOUBLE LINKED LIST =====
1. Tambah data di awal
2. Tambah data di akhir
3. Sisipkan data di tengah (setelah NIM)
4. Hapus data di awal
5. Hapus data di akhir
6. Tampilkan data
7. Tampilkan data dari belakang
0. Keluar
Pilih menu : 0
Program selesai.

```

12.3.3 Pertanyaan Percobaan

1. Perhatikan potongan kode berikut pada method `removeFirst()`:

```
head = head.next;
```

```
head.prev = null;
```

Jelaskan fungsi masing-masing statement tersebut pada proses penghapusan node!

Jawaban :

- `head = head.next;` Berfungsi untuk memindahkan *pointer* head ke *node* urutan kedua. Karena *node* pertama akan dihapus, maka *node* di belakangnya harus naik menjadi *node* pertama (*head* yang baru).
- `head.prev = null;` Berfungsi untuk memutus rantai dari *head* yang baru kembali ke *node* yang lama. pada *Double Linked List*, *node* pertama penunjuk *prev*-nya harus null. Dengan memberikan nilai null, *node* lama benar-benar terputus dari *list* dan nantinya akan otomatis dihapus.

2. Modifikasi method removeFirst() dan removeLast() agar program menampilkan data yang berhasil dihapus!

Jawaban :

```
public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    mahasiswa24 removedData = head.data;
    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
    System.out.println("Data mahasiswa dengan NIM " + removedData.nim + " bernama " + removedData.nama
+ " berhasil dihapus.");
}

public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    mahasiswa24 removedData = tail.data;

    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }
    System.out.println("Data mahasiswa dengan NIM " + removedData.nim + " bernama " + removedData.nama
+ " berhasil dihapus.");
}
```

CM2

- **Pembeli24.java**

Code :

```
package cm2;

public class pembeli24 {
    int noantrian;
    String namapembeli;
    String nohp;

    public pembeli24(int noantrian, String namapembeli, String nohp) {
        this.noantrian = noantrian;
        this.namapembeli = namapembeli;
        this.nohp = nohp;
    }
}
```

- **Pesanan24.java**

Code :

```
package cm2;

public class pesanan24 {
    int kodepesanan;
    String namapesanan;
    int harga;

    public pesanan24(int kodepesanan, String namapesanan, int harga) {
        this.kodepesanan = kodepesanan;
        this.namapesanan = namapesanan;
        this.harga = harga;
    }
}
```

- **Nodepembeli24.java**

Code :

```
package cm2;

public class nodepembeli24 {
    pembeli24 data;
    nodepembeli24 prev, next;

    public nodepembeli24(nodepembeli24 prev, pembeli24 data, nodepembeli24 next) {
        this.prev = prev;
        this.data = data;
        this.next = next;
    }
}
```

- **Nodepesanan24.java**

Code :

```
package cm2;

public class nodepembeli24 {
    pembeli24 data;
    nodepembeli24 prev, next;

    public nodepembeli24(nodepembeli24 prev, pembeli24 data, nodepembeli24 next) {
        this.prev = prev;
        this.data = data;
    }
}
```

```

    this.next = next;
}
}

```

- **Doublelinkedlistpembeli24.java**

Code :

```

package cm2;

public class doublelinkedlistpembeli24 {
    nodepembeli24 head, tail;
    int size;

    public boolean isEmpty() {
        return head == null;
    }

    // Enqueue (Tambah di akhir)
    public void addLast(pembeli24 data) {
        if (isEmpty()) {
            head = tail = new nodepembeli24(null, data, null);
        } else {
            nodepembeli24 newNode = new nodepembeli24(tail, data, null);
            tail.next = newNode;
            tail = newNode;
        }
        size++;
    }

    public pembeli24 removeFirst() throws Exception {
        if (isEmpty()) {
            throw new Exception("Antrian Kosong!");
        }
        pembeli24 data = head.data;
        if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
            head.prev = null;
        }
        size--;
        return data;
    }

    public void print() {
        if (isEmpty()) {
            System.out.println("Antrian Kosong!");
            return;
        }
        System.out.println("=====");
        System.out.println("Daftar Antrian pembeli24");
        System.out.println("=====");
        System.out.printf("%-15s %-15s %-15s\n", "No Antrian", "Nama", "No HP");
        nodepembeli24 current = head;
        while (current != null) {
            System.out.printf("%-15d %-15s %-15s\n", current.data.noantrian, current.data.namapembeli,
current.data.nohp);
            current = current.next;
        }
    }
}

```

